Objective

The goal of this project is to **design, implement, and analyze** a parallel high-performance simulation of the **Convolution Operation program.** Starting from the sequential implementation of the problem, students will parallelize it using three different parallel programming paradigms:

- **OpenMP** for shared-memory parallelism,
- **Hybrid (MPI-OpenMP)** for distributed-memory parallelism, and
- **CUDA** for GPU-based parallelism.

With each implementation, students will analyze the impact of different parallelization strategies on performance, scalability and implementation complexity.

Description of the Problem

Convolution [1] is a *mathematical* operation on two functions **f** and **g** used to produce a third function that is interpreted as a modified version of one of the original functions. Basically, it gives the area overlapped between both functions as a function of the amount that one of the original functions is translated, as it is shown in Figure 1.

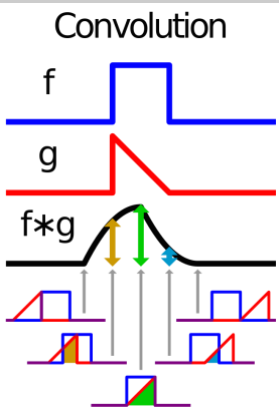

Figure 1 Convolution process

Convolution is used on probability, statistics, computer vision, natural language processing, image and signal processing, engineering, and differential equations.

To clarify the objectives of the convolution process, it will be presented applied in the field of image processing. In this field, the convolution process is widely used to change the intensity of each individual pixel to reflect the intensities of the surrounding area.

A common use of convolution is to create image filters as could be Blur, Sharpen, Emboss and Edge detection. Figure 2 shows the results after applying the emboss filter.

Figure 2 Emboss effect

The convolution process in the field of image processing is based on applying a

matrix named as “convolution kernel”. This matrix consists of an *n×n* matrix identifying the surrounding area of the evaluated pixel. Each cell of the kernel contains a pixel weight in respect of the pixel.

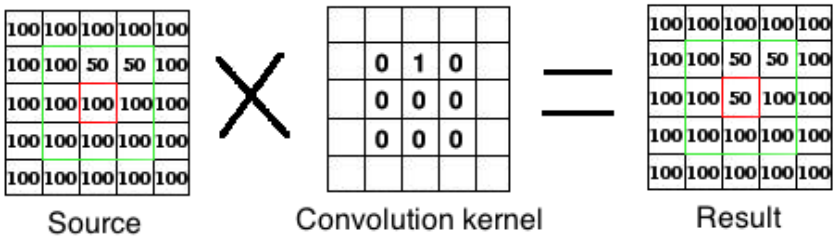

Figure 3 3x3 Convolution Kernel. The red cell represents the evaluated pixel.

In the example shown in Figure 3, the convolution process corresponds to the individual multiplication of every paired cells/pixels, from the source and the kernel. Finally, it obtains the sum of all of them, which corresponds to the resulting pixel value, as it is presented in the Figure 4.

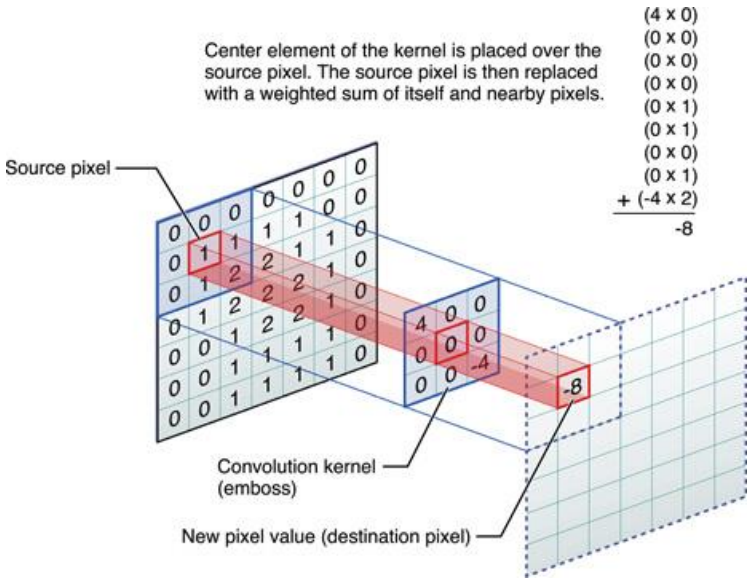

Figure 4 Convolution process

Varying the composition of the kernel, it is possible to obtain different results. Figure 5 shows different image convolution results.

Original picture

Blur | Motion blur | Finding edges | Sharpen

Figure 5 Convolution operations with different kernel matrix.

It is also possible to have more complex functions than the simple *add* to achieve different results.

The convolution process complexity depends on the original matrix size, the kernel dimension and on the mathematical function applied to the cells. By this reason, its parallelization is an important technique to consider in further implementations.

Serial Code

To compare the performance and the results obtained from different groups solutions, we will provide the test-bed that you must pass for providing your results and elaborate the reports. The material you have for this project is as follows:

- **Serial code**: Source code for a serial implementation. This serial implementation allows you to process small and large images. For large images (>500MB) you will have problems allocating enough memory for processing convolution. If you want to test for images bigger than the images provided for us, then you would need to split the original images into partitions. The serial code provided, the number of partitions is an argument of the program.
- **PPM images**: In folder "*/share/apps/files/convolution/images*", located into each node of the cluster moore, you can find the set of images to test. Don't copy these images into your $HOME folder given that disk space at the front-end is scarce and will be exhausted very soon.
- **Kernel matrix**: In folder "*/share/apps/files/convolution/kernel*" of the cluster *moore*, you can find the set of kernel matrices. Kernel matrices are described using text files. The kernel provided *5x5 Sharpen* is very useful in image processing area; the big ones (25x25, 49x49, 99x99) are randomly generated for testing the scalability of your proposals.
- The serial code generates a text file with the time spent in different execution phases. The Figure 6 shows an example of the output of the serial convolution program.

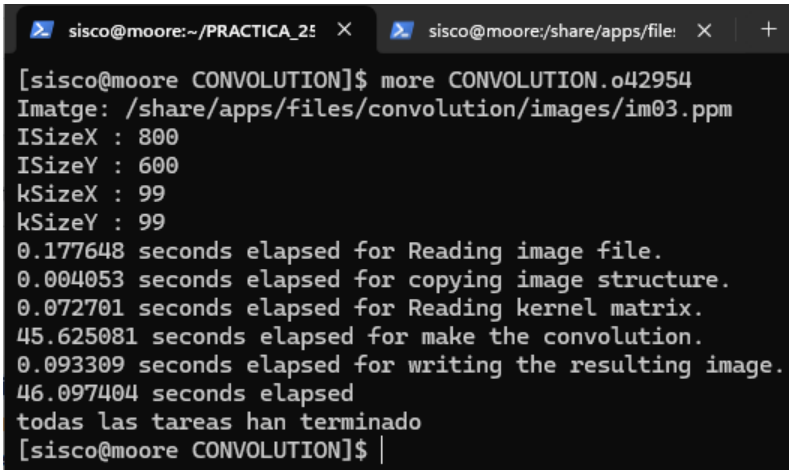

Figure 6. Screenshot of the serial convolution output.

In addition, the serial code generates a PPM image as the result of the convolution between the original image and the selected kernel. You may store the resulting images in your $HOME directory; however, your disk quota may be exceeded quickly. Therefore, we strongly recommend transferring these files to your laptop using the sftp command and then deleting them from the cluster.

Source code compilation and execution

- For the serial source code compilation, you should execute as follows the Figure 7.

```
$gcc convolution.c -o serialconv
$./serialconv
share/apps/files/convolution/image/source_image.ppm
share/apps/files/convolution/kernel/kernel.txt
output_image.ppm number_partitions
```

Figure 7. Arguments of the serial convolution program.

Remember that is **totally forbidden to execute the code in the front-end** of the cluster. Doing this, you can collapse the server or produce a server downfall. By this, it is important to use the front-end queuing system even for serial programs.

For executing the serial code into the *moore cluster*, you can use the same script used with the serial NAS parallel benchmarks together with the *qsub command*. See Figure 8.

```
$qsub run-simple-serial.sh
```

Figure 8. Execute the serial code into the cluster.

To have serial execution times as a reference for future parallelization versions, you should calculate the execution time of the serial convolution program for arguments given on Table 1.

Image	Kernel			
	kernel5x5_Sharpen.txt	Kernel25x25_Random2.txt	Kernel49x49_Random2txt	Kernel99x99-Random2.txt
im01.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im02.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im03.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im04.ppm	Partitions=1	Partitions=1	Partitions=1	Partitions=1
im05.ppm	Partitions=3	Partitions=3	Partitions=3	Partitions=3

Table 1. Table with arguments to calculate the elapsed time times of the serialconv.

Note that the combination of ima05.ppm with any kernel must be executed with at least three partitions. Also, keep in mind that serial execution with the im05.ppm image and the 49x49 and kernel99x99 kernels can take over 2000s and 10000s, respectively.

Project Statement: OpenMP parallelization practice

Delivery 1: Implementation with OpenMP (Deadline 19th of April)

To obtain the best performance of an application implemented with a parallel programming model, firstly, we should start by optimizing the application at node-level. According to this, the first delivery will focus on studying the operation of the sequential version, analysing the pieces of code liable to be parallelized following a work decomposition model. Next, we will apply the OpenMP directives [2] to parallelize the corresponding code according to the hardware and the suitable work decomposition model at node level.

When developing this parallel version of the application, it is necessary to consider the following challenges: dependencies, memory traffic, computational cost and their synchronization needs.

To measure the time in the OpenMP programs, you must use the *omp_get_wtime()* function, as Figure 9 shows.

```
double start_time, elapsed_time;
int x=100,y=25,z;
start_time=omp_get_wtime();
z=x*y;
elapsed_time=omp_get_wtime()-start_time;
```

Figure 9. Example of omp_get_wtime().

To execute the OpenMP parallel version, remember that you must use the script *run-simple-omp.sh* provided into the Benchmarking activity. All the scripts needed to develop the project can be found in "/share/apps/files/convolution/scripts" folder of the moore cluster.

Remember that this delivery has a weight of 25% over the total mark of the subject.

Material to deliver

Students must deliver the source code. The code will be delivered using a *github repository*. The code should contain explanations of the main decisions and library functions used to parallelize the problem. A ***github*** reference to the code must be included in the corresponding report.

You should also do a brief PDF report (maximum 8 pages) with the following items:

I. **Main decisions taken to parallelize the problem**: You should explain how you have parallelized the problem from the serial version. Analyse the serial code and detect the code susceptible to be parallelized. Justify the strategy used to parallelize the serial code. You should point out how has solved the challenges cited above. If you have tested different strategies, you can describe briefly each of them and justify the reason of your choice.

II. **Scalability of the program**: You should show the *Execution Time*, together with the *Speedup and Efficiency* for different number of processes/cores and problem size (image and kernel). It is better if you use figures instead of tables.

III. **Discussion of the obtained results:** You should explain the results for the main metrics (*Speedup* and *Efficiency*) and the reasons for the obtained results. Consider on your discussion the effects of the dependences, synchronization and memory traffic between data, in case it was needed, in your solution.

IV. **Remember that the report extension should not overcome 8 pages. If you need more pages, you can put the least significant results into annexes.**

Delivery Instructions

To do this practice, you need to have an user account in EPS teaching cluster (*moore.udl.cat*) and install in your local computer a *ssh* client to connect to the cluster. If you connect from outside of the UdL network, you need a VPN connection. To do this, you need to install a *FortiClient* in your laptop and the *Authenticator* app in your mobile for a 2FA authentication process. A manual with the instructions to install both can be found in the folder "/Resources/VPN" of the virtual campus.

In the cluster, you will have all the additional software that you need to do the project.

Likewise, to develop and test the code on your local computer, you will need to install any software that implements OpenMP version 3.

To run the serial code for this practice, there are no special requirements. You can find the source code in the front-end of the *moore.udl.cat* cluster, folder "*/share/apps/files/heat/code*" and also in the Project folder that you can find in Resources section of the virtual campus (/*Resources/Project_HPC*).

You should deliver reports throughout the Sakai virtual campus, by means of the corresponding *Activity Section*. **All the activities can be done individually or by groups of two students.**

To make the report understandable and easily evaluated or reviewed by any reader, you must follow the rules of good practices to prepare a report. The report must contain the title of the activity and the authors. Use the editing techniques properly: tables, figures, cross-references, table of contents, bibliography etc. The report should be complete, justifying the design decisions, making a proper analysis of the performance and the comparison of the effects of different alternatives to reach the more appropriate decision. You can insert some code pieces for helping to explain your solutions. The figures must be correctly formatted considering aspects such as titles, axis labels, legend, type of lines, symbols, colours, etc. The text in the figures, labels and axis must be legible. The axis must be correctly dimensioned. The type of chart must be selected correctly depending on that you want to explain. Figures must be accompanied by a brief explanation.

Some common mistakes that you should avoid:

- Vague explanations without justification.
- Explanations of the implementation without justify the structure and the decisions you took.
- Lots of tables with data. You must identify the most important data and summarize your explanations. Be concrete. Add value with your contributions.
- The figures are not properly created, bad axis ranges, no labels, blurry, or added as screenshot.
- The figures are not representative, lots of figures that can be summarized, etc.

Bibliography

1.- Gonzalez, R. C. and Woods, R. E. *Digital Image Processing*. Prentice Hall, 2008.

2.- Chapman, B., Jost, G., and Van der Pas, R. *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA, USA: MIT Press, 2007.